PROJECT 3 NOTES

E-Commerce SQL Analytics

UCI Online Retail Dataset — 541,909 Transactions

Madhunika Danam | Connecticut, USA | June 2026

541,909	38	£9.7M	21 Days
Transactions	Countries	Total Revenue	Learning Journey

Table of Contents

Section	Topic	Page
1	Project Overview & Setup	2
2	Week 1 — SQL Foundations	3
3	Week 2 — JOINs, Subqueries, CASE WHEN, Dates	5
4	Week 3 — CTEs, Window Functions, LAG/LEAD	8
5	Python + SQLite Integration	11
6	Full Business Analysis	12
7	Key Business Insights & Recommendations	14
8	Interview Q&A Guide	15
9	SQL Quick Reference Card	17

1. Project Overview & Setup

This project analyzes 541,909 real e-commerce transactions from a UK-based online gift shop using SQL, Python, SQLite, Pandas, and Matplotlib. The goal was to answer real business questions like a Data Analyst.

Property	Value
Source	UCI Machine Learning Repository — Online Retail Dataset
Transactions	541,909 rows | 8 columns
Period	December 2010 — December 2011
Company	Real UK-based online gift shop
Countries	38 countries worldwide
Database	SQLite (ecommerce.db)
File Path	C:\Users\danam\OneDrive\Desktop\ML_journey\project_03_ecommerce_sql

Database Tables Created

Table	How Created	Contents
orders	Day 1 — from CSV	541,909 rows — all transactions
country_info	Day 8 — manually inserted	7 countries with Region and Currency
country_summary	Day 18 — pd.to_sql()	Top 5 countries by revenue
monthly_summary	Day 18 — pd.to_sql()	Monthly revenue — 13 months

Setup Cell — Always Run First!

```
import sqlite3 import pandas as pd import matplotlib.pyplot as plt import seaborn as sns pd.set_option('display.float_format', '{:.2f}'.format) conn = sqlite3.connect( r"C:\Users\danam\OneDrive\Desktop\ML_journey\ project_03_ecommerce_sql\datasets\ecommerce.db") conn.execute("DELETE FROM country_info") conn.executemany("INSERT INTO country_info VALUES (?, ?, ?)", [ ('United Kingdom', 'Europe', 'GBP'), ('Germany', 'Europe', 'EUR'), ('France', 'Europe', 'EUR'), ('Netherlands', 'Europe', 'EUR'), ('Australia', 'Oceania', 'AUD'), ('Japan', 'Asia', 'JPY'), ('USA', 'Americas', 'USD') ]) conn.commit() print("Setup complete!")
```

Important Rules:

- Always use query_q1, query_q2 naming — never just 'query' to avoid overwriting!
- Always Kernel → Restart & Run All when reopening notebook!
- Always add WHERE Quantity > 0 to remove cancellations!
- Always add CustomerID IS NOT NULL for customer analysis!
- Date format is DD/MM/YYYY — ALWAYS use substr() not strftime()!
- substr(InvoiceDate, 7, 4) = year | substr(InvoiceDate, 4, 2) = month

2. Week 1 — SQL Foundations

What is SQL?

SQL = Structured Query Language. It is the language you use to talk to a database. A database is like an organized Excel file that can hold millions of rows and multiple tables.

Week 1 Commands Reference

Command	What it does	Python Equivalent
SELECT	Choose columns to show	df[['col1','col2']]
FROM	Choose which table	pd.read_csv('file.csv')
WHERE	Filter rows	df[df['col'] > 0]
GROUP BY	Group rows for aggregation	df.groupby('col')
ORDER BY DESC	Sort highest first	df.sort_values(ascending=False)
LIMIT	Show only N rows	df.head(N)
COUNT(*)	Count all rows	df.shape[0]
SUM()	Add up values	df['col'].sum()
AVG()	Calculate average	df['col'].mean()
MIN/MAX()	Smallest/largest value	df['col'].min()/.max()
DISTINCT	Unique values only	df['col'].nunique()
AS	Rename a column	df.rename(columns={...})
HAVING	Filter groups after GROUP BY	No direct equivalent

SQL Execution Order — MEMORIZE THIS!

Step	Command	What happens
1	FROM	Get all rows from table
2	WHERE	Filter individual rows
3	GROUP BY	Group remaining rows
4	HAVING	Filter groups
5	SELECT	Choose columns + calculate
6	ORDER BY	Sort results
7	LIMIT	Restrict number of rows

WHERE vs HAVING

	WHERE	HAVING
When it runs	BEFORE GROUP BY	AFTER GROUP BY
Filters	Individual rows	Groups of rows
Can use aggregates?	NO — SUM/COUNT not allowed	YES — SUM/COUNT allowed
Example	WHERE Quantity > 0	HAVING COUNT(*) > 1000

Week 1 Key Findings

- Total revenue = £9.7M (includes dirty data with cancellations)
- UK = 84% of total revenue
- Netherlands = £120 avg order value (highest!)
- REGENCY CAKESTAND = star product at £164,762
- 135,080 orders missing CustomerID (guest orders = 24.93%)
- 4,372 unique registered customers
- 11 countries have 1,000+ orders
- Only 13 products earn above £50,000 — star products!

3. Week 2 — JOINs, Subqueries, CASE WHEN, Date Functions

JOINs Explained

JOINs combine two tables together based on a matching column. We created country_info table with 7 countries and joined it with orders table.

JOIN Type	What it keeps	Use case
INNER JOIN	Only rows that MATCH in BOTH tables	Only want exact matches
LEFT JOIN	ALL rows from left table + matches from right	Keep all data, add extra info
RIGHT JOIN	ALL rows from right table + matches from left	Rarely used in practice

```
-- INNER JOIN — only 7 countries show (matching ones only!) SELECT a.Country, a.Revenue, b.Region, b.Currency FROM orders a INNER JOIN country_info b ON a.Country = b.Country -- LEFT JOIN — all 38 countries show! NULL where no match SELECT a.Country, a.Revenue, b.Region FROM orders a LEFT JOIN country_info b ON a.Country = b.Country
```

Key Insight: LEFT JOIN revealed 31 countries INNER JOIN was hiding! Always use LEFT JOIN when you want ALL data.

Subqueries

A subquery is a query inside another query. The inner query runs FIRST, then the outer query uses the result. Think of it like Russian dolls!

```
-- Find countries above average revenue SELECT Country, total_revenue FROM ( SELECT Country, SUM(Quantity * UnitPrice) AS total_revenue FROM orders GROUP BY Country ) WHERE total_revenue > ( SELECT AVG(Quantity * UnitPrice) FROM orders )
```

Key Insight: Only 3 countries above average revenue: UK, Netherlands, EIRE

CASE WHEN — if/else Logic in SQL

CASE WHEN is exactly like if/elif/else in Python. Use it to create new category columns.

```
-- Python equivalent: -- if revenue > 1000000: tier = 'Top Performer' -- elif revenue > 100000: tier = 'Mid Performer' -- else: tier = 'Low Performer' SELECT Country, total_revenue, CASE WHEN total_revenue > 1000000 THEN 'Top Performer' WHEN total_revenue > 100000 THEN 'Mid Performer' ELSE 'Low Performer' END AS performance_tier FROM country_revenue
```

Key Insight: Only UK = Top Performer. 5 countries = Mid Performer. 32 countries = Low Performer.

Date Functions — CRITICAL for this dataset!

WARNING: This dataset uses DD/MM/YYYY HH:MM:SS format. strftime() does NOT work. Always use substr()!

What you need	WRONG (strftime)	CORRECT (substr)

Extract Year	strftime('%Y', InvoiceDate)	substr(InvoiceDate, 7, 4)
Extract Month	strftime('%m', InvoiceDate)	substr(InvoiceDate, 4, 2)
Extract Day	strftime('%d', InvoiceDate)	substr(InvoiceDate, 1, 2)
Year-Month	strftime('%Y-%m', InvoiceDate)	substr(InvoiceDate, 7, 4) || '-' || substr(InvoiceDate, 4, 2)

Key Insight: November 2011 = peak month at £1,509,496. Thursday = busiest day. Saturday = ZERO orders (shop closed!).

4. Week 3 — CTEs, Window Functions, LAG/LEAD

CTEs — Common Table Expressions

CTE = temporary named table — like a Python variable that stores a query result. Makes complex queries readable and reusable!

```
-- Single CTE WITH country_revenue AS ( SELECT Country, ROUND(SUM(Quantity * UnitPrice), 2) AS total_revenue FROM orders WHERE Quantity > 0 GROUP BY Country ) SELECT * FROM country_revenue ORDER BY total_revenue DESC LIMIT 5 -- Two CTEs + JOIN WITH country_revenue AS ( SELECT Country, ROUND(SUM(Quantity * UnitPrice), 2) AS total_revenue FROM orders WHERE Quantity > 0 GROUP BY Country ), country_orders AS ( SELECT Country, COUNT(*) AS total_orders FROM orders WHERE Quantity > 0 GROUP BY Country ) SELECT country_revenue.Country, total_revenue, total_orders FROM country_revenue JOIN country_orders ON country_revenue.Country = country_orders.Country ORDER BY total_revenue DESC
```

Window Functions

Unlike GROUP BY which collapses rows, Window Functions ADD a new column to ALL existing rows without losing any data!

Function	What it does	Example result
ROW_NUMBER()	Always unique 1,2,3,4,5 — ignores ties!	1, 2, 3, 4, 5
RANK()	Same number for ties — SKIPS next!	1, 2, 2, 4, 5
DENSE_RANK()	Same number for ties — NEVER skips!	1, 2, 2, 3, 4
PARTITION BY	Resets rank for each group — rows stay!	Rank within country

```
-- Global ranking DENSE_RANK() OVER (ORDER BY total_spent DESC) AS customer_rank -- Ranking within each country (PARTITION BY!) DENSE_RANK() OVER ( PARTITION BY Country ORDER BY total_spent DESC ) AS country_rank -- Filter on window function result — needs subquery! SELECT * FROM ( SELECT Country, CustomerID, total_spent, DENSE_RANK() OVER (PARTITION BY Country ORDER BY total_spent DESC) AS country_rank FROM customer_spending ) WHERE country_rank = 1 -- top spender per country!
```

Key Insight: Customer 14646 = global VIP at £280,206! Netherlands top spender is also global #1!

LAG and LEAD Functions

LAG looks at the PREVIOUS row. LEAD looks at the NEXT row. Perfect for month-on-month comparison and trend analysis!

Function	Looks	First/Last row	Use case
LAG(col, 1)	Backwards at previous row	First row = NULL	What was last month?
LEAD(col, 1)	Forwards at next row	Last row = NULL	What will next month be?

```
WITH monthly_revenue AS ( SELECT substr(InvoiceDate, 7, 4) || '-' ||
substr(InvoiceDate, 4, 2) AS month, ROUND(SUM(Quantity * UnitPrice), 2) AS revenue
FROM orders WHERE Quantity > 0 GROUP BY month ) SELECT month, revenue, LAG(revenue, 1)
OVER (ORDER BY month) AS prev_revenue, ROUND(revenue - LAG(revenue, 1) OVER (ORDER BY
month), 2) AS growth, LEAD(revenue, 1) OVER (ORDER BY month) AS next_revenue FROM
monthly_revenue ORDER BY month
```

Key Insight: November 2011 = +£354,517 biggest monthly jump! December = -£870,703 biggest drop! Classic Christmas gift shop pattern.

5. Python + SQLite Integration

Connection Methods

Method	What it does	Example
sqlite3.connect()	Opens connection to database	conn = sqlite3.connect('db.db')
pd.read_sql_query()	Run SQL, return DataFrame	result = pd.read_sql_query(query, conn)
conn.execute()	Run one SQL statement	conn.execute('DELETE FROM table')
conn.executemany()	Run SQL for many rows	conn.executemany('INSERT...', data)
conn.commit()	Save changes permanently	conn.commit()
df.to_sql()	Write DataFrame to database	df.to_sql('table', conn, if_exists='replace')
conn.close()	Close connection	conn.close()

pd.to_sql() Parameters

```
df_summary.to_sql( 'country_summary', # name of table in database conn, # which database connection if_exists='replace', # 'replace' / 'append' / 'fail' index=False # don't save pandas row numbers )
```

sqlite_master — See All Tables

```
-- See ALL tables in your database pd.read_sql_query(""" SELECT name FROM sqlite_master WHERE type='table' """, conn) -- Shows: orders, country_info, country_summary, monthly_summary
```

Key Insight: sqlite_master is a hidden directory table in every SQLite database — like a library catalog!

6. Full Business Analysis — Day 20 Queries

Query 1 — VIP Customer Ranking

```
WITH customer_spending AS ( SELECT CustomerID, ROUND(SUM(Quantity * UnitPrice), 2) AS total_spent FROM orders WHERE Quantity > 0 AND CustomerID IS NOT NULL GROUP BY CustomerID ) SELECT CustomerID, total_spent, DENSE_RANK() OVER (ORDER BY total_spent DESC) AS customer_rank FROM customer_spending ORDER BY total_spent DESC LIMIT 10
```

Key Insight: Customer 14646 = #1 VIP at £280,206! Top 10 customers need loyalty program immediately!

Query 2 — Monthly Revenue with Growth

```
WITH monthly_revenue AS ( SELECT substr(InvoiceDate, 7, 4) || '-' || substr(InvoiceDate, 4, 2) AS month, ROUND(SUM(Quantity * UnitPrice), 2) AS revenue FROM orders WHERE Quantity > 0 GROUP BY month ) SELECT month, revenue, LAG(revenue, 1) OVER (ORDER BY month) AS prev_revenue, ROUND(revenue - LAG(revenue, 1) OVER (ORDER BY month), 2) AS growth FROM monthly_revenue ORDER BY month
```

Key Insight: November peak +£354,517. December crash -£870,703. Stock up by September every year!

Query 3 — Country Performance Tiers

```
WITH country_revenue AS ( SELECT Country, ROUND(SUM(Quantity * UnitPrice), 2) AS total_revenue FROM orders WHERE Quantity > 0 GROUP BY Country ) SELECT cr.Country, total_revenue, CASE WHEN total_revenue > 1000000 THEN 'Top Performer' WHEN total_revenue > 100000 THEN 'Mid Performer' ELSE 'Low Performer' END AS performance_tier, ci.Region FROM country_revenue cr LEFT JOIN country_info ci ON cr.Country = ci.Country ORDER BY total_revenue DESC
```

Key Insight: Only UK = Top Performer! 5 Mid, 32 Low. Dangerous concentration in one country!

Query 4 — Top Products by Revenue

```
SELECT Description, ROUND(SUM(Quantity * UnitPrice), 2) AS total_revenue FROM orders WHERE Quantity > 0 GROUP BY Description ORDER BY total_revenue DESC LIMIT 10
```

Key Insight: DOTCOM POSTAGE is shipping not a product! REGENCY CAKESTAND is true star at £174,484!

7. Key Business Insights & Recommendations

Area	Finding	Recommendation
Revenue	Total £9.7M | Peak November £1.5M | December crash -£870K	Stretch goal: £10.7M by September! Launch August marketing campaig
Customers	Customer 14646 = £280K VIP | 24.93% guest checkouts | 871 one-time buyers	Launch VIP loyalty program and incentivize guest registration!
Countries	UK = 89.9% of revenue — dangerous concentration!	Diversify! Target EIRE, Netherlands + Germany aggressively!
Products	REGENCY CAKESTAND = £174K star! DOTCOM POSTAGE = £77K shipping not product!	Promote CAKESTAND as hero! Clean postage from produc
Australia	Only 1,185 orders but £138K revenue — high value hidden gem!	Invest in Australian market — high order value opportunity!

Priority Action Plan

Priority	Action	Business Impact
HIGH	Launch VIP loyalty program	Retain £280K+ customers
HIGH	Diversify beyond UK market	Reduce 89.9% concentration risk
HIGH	Convert guest checkouts	Capture 24.93% more customer data
MEDIUM	August marketing campaign	Capture Christmas buildup early
MEDIUM	Invest in Australian market	High value hidden gem opportunity
LOW	Clean product data	Remove shipping charges from reports

8. Interview Q&A; Guide

Q: Tell me about your SQL project.

A: I analyzed 541,909 real e-commerce transactions from a UK gift shop using SQL, Python, SQLite, Pandas and Matplotlib. I progressed from basic SELECT queries to advanced CTEs, Window Functions and LAG/LEAD trend analysis. My biggest finding was that 89.9% of revenue comes from UK alone — a dangerous concentration risk I flagged immediately.

Q: What is the difference between WHERE and HAVING?

A: WHERE filters individual rows before grouping — it cannot use aggregate functions like SUM or COUNT because groups don't exist yet. HAVING filters groups after GROUP BY — it can use aggregate functions. For example: WHERE Quantity > 0 removes individual cancellations, while HAVING COUNT(*) > 1000 removes small country groups.

Q: What is a CTE and why use it?

A: CTE stands for Common Table Expression — it creates a temporary named table using the WITH clause. It is like a Python variable that stores a query result. I use CTEs to make complex queries readable, avoid repeating code, and break multi-step analysis into clean logical blocks.

Q: What is the difference between RANK and DENSE_RANK?

A: Both give the same number to tied rows. But RANK skips the next number after a tie — like 1,2,2,4. DENSE_RANK never skips — like 1,2,2,3. For business ranking I prefer DENSE_RANK because it gives fair sequential positions without gaps.

Q: What is a Window Function?

A: A Window Function calculates across rows without collapsing them, unlike GROUP BY. It adds a new calculated column to every row while keeping all rows intact. I used DENSE_RANK, ROW_NUMBER, LAG and LEAD as window functions in my project.

Q: What did LAG help you discover?

A: LAG helped me do month-on-month revenue analysis. By subtracting LAG(revenue, 1) from current revenue, I calculated growth for each month. I discovered November 2011 had the biggest single month jump of +£354,517, and December had the biggest drop of -£870,703 — confirming the classic Christmas gift shop seasonal pattern.

Q: Tell me about a data quality issue you found.

A: DOTCOM POSTAGE appeared as the top product by revenue at £206,248 — but it is actually a shipping charge, not a real product. Similarly, 'Manual' and 'POSTAGE' entries polluted the product analysis. As a Data Analyst I immediately flagged these for removal from product reports. This shows the importance of always questioning data before drawing conclusions.

Q: What was your most interesting business insight?

A: Netherlands had only 2,371 orders compared to Germany's 9,495 — but Netherlands generated more revenue at £285,446 versus Germany's £228,867. This means Netherlands customers spend £120 per order on average compared to only £25 for UK customers — 5 times more valuable per transaction! I recommended the business invest aggressively in the Dutch market.

9. SQL Quick Reference Card

Week 1 — Foundations

Command	Syntax	Purpose
SELECT	SELECT col1, col2 FROM table	Choose columns
WHERE	WHERE Quantity > 0	Filter rows
GROUP BY	GROUP BY Country	Group for aggregation
HAVING	HAVING COUNT(*) > 1000	Filter groups
ORDER BY	ORDER BY revenue DESC	Sort results
COUNT	COUNT(*) / COUNT(DISTINCT col)	Count rows/unique
SUM/AVG	SUM(Quantity * UnitPrice)	Aggregate math

Week 2 — Joins & Logic

Command	Syntax	Purpose
INNER JOIN	FROM a INNER JOIN b ON a.col = b.col	Only matching rows
LEFT JOIN	FROM a LEFT JOIN b ON a.col = b.col	All left + matches
Subquery	WHERE col > (SELECT AVG(col) FROM table)	Query inside query
CASE WHEN	CASE WHEN x>100 THEN 'High' ELSE 'Low' END	if/else logic
substr date	substr(InvoiceDate, 7, 4) || '-' || substr(InvoiceDate, 4, 2)	Year-Month

Week 3 — Advanced Analytics

Command	Syntax	Purpose
CTE	WITH name AS (SELECT...) SELECT * FROM name	Temp named table
ROW_NUMBER	ROW_NUMBER() OVER (ORDER BY col DESC)	Unique row numbers
RANK	RANK() OVER (ORDER BY col DESC)	Rank with gaps
DENSE_RANK	DENSE_RANK() OVER (ORDER BY col DESC)	Rank no gaps
PARTITION BY	DENSE_RANK() OVER (PARTITION BY grp ORDER BY col)	Rank within groups
LAG	LAG(col, 1) OVER (ORDER BY date_col)	Previous row value
LEAD	LEAD(col, 1) OVER (ORDER BY date_col)	Next row value
Growth	col - LAG(col,1) OVER (ORDER BY date_col)	Period change
to_sql	df.to_sql('table', conn, if_exists='replace', index=False)	Write to DB

Key Numbers to Remember

Metric	Value
Total Transactions	541,909 rows
Countries	38 countries
Total Revenue	£9.7M
UK Revenue Share	89.9%
Peak Month	November 2011 — £1,509,496
Lowest Month	February 2011 — £523,631
Top VIP Customer	Customer 14646 — £280,206
Netherlands Avg Order	£120 vs UK £25
Guest Checkout Rate	24.93%
Registered Customers	4,372
Above Average Spenders	871 customers (20%)
Star Product	REGENCY CAKESTAND — £174,484
Top Performer Countries	1 (UK only!)
Mid Performer Countries	5
Low Performer Countries	32